



Exploiting recurrent graph neural networks for suffix prediction in predictive monitoring

Efrén Rama-Maneiro¹ · Juan C. Vidal^{1,2} · Manuel Lama¹ · Pablo Monteagudo-Lago¹

Received: 3 April 2024 / Accepted: 24 June 2024 / Published online: 9 July 2024

© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2024

Abstract

Predictive monitoring is a subfield of process mining that aims to predict how a running case will unfold in the future. One of its main challenges is forecasting the sequence of activities that will occur from a given point in time —suffix prediction—. Most approaches to the suffix prediction problem learn to predict the suffix by learning how to predict the next activity only, while also disregarding structural information present in the process model. This paper proposes a novel architecture based on an encoder-decoder model with an attention mechanism that decouples the representation learning of the prefixes from the inference phase, predicting only the activities of the suffix. During the inference phase, this architecture is extended with a heuristic search algorithm that selects the most probable suffix according to both the structural information extracted from the process model and the information extracted from the log. Our approach has been tested using 12 public event logs against 6 different state-of-the-art proposals, showing that it significantly outperforms these proposals.

Keywords Process mining · Predictive monitoring · Recurrent neural networks · Encoder-decoder models · Attention mechanisms

✉ Efrén Rama-Maneiro
efren.rama.maneiro@usc.es

Juan C. Vidal
juan.vidal@usc.es

Manuel Lama
manuel.lama@usc.es

Pablo Monteagudo-Lago
pablo.monteagudo.lago@rai.usc.es

¹ Centro Singular de Investigación en Tecnoloxías Intelixentes (CiTIUS), Universidade de Santiago de Compostela, Santiago de Compostela, Spain

² Departamento de Electrónica e Computación, Universidade de Santiago de Compostela, Santiago de Compostela, Spain

Mathematics Subject Classification 68T07 · 05C90

1 Introduction

Process mining has emerged as a discipline that aims to discover, monitor, and improve business processes by extracting knowledge from past execution data that are recorded in an event log [1]. One of the process mining tasks that has received a lot of attention in the past years is forecasting how a case is going to unfold. This allows organizations to prevent undesired outcomes, issues, or delays [2]. This task belongs to a subfield of process mining, known as *predictive monitoring*, which leverages machine learning techniques in order to learn a predictive model over the event log. Many predictive problems can be defined, such as forecasting the next activity, the next timestamp, the remaining time until the end of the case, or the next sequence of activities. This paper focuses on the latter problem, aka activity suffix prediction.

Different machine learning techniques have been used in predictive monitoring [3–6], but deep learning still holds the best overall results [7]. Among these latter techniques, recurrent neural networks (RNNs) are one of the most used models due to their adequacy to the sequential nature of business process logs. However, other types of deep neural networks, such as Convolutional Neural Networks (CNNs) [8, 9], Graph Neural Networks (GNNs) [10, 11], Generative Adversarial Networks (GANs) [12], or autoencoders [13] have also achieved competitive results. Regarding the suffix prediction problem, several deep learning approaches have been proposed, either based on encoder-decoder architectures [12, 14–16], or not [14, 17–26].

On the one hand, approaches based on encoder-decoder architectures do not truly exploit the capabilities of these types of neural models with attention mechanisms. The encoder and decoder of these models are trained to predict the same features so they still have the aforementioned issues, with the exception of [15] approach, which does not learn to predict the suffix but the next activity, so it does not have information from the whole target suffix during the training phase. In the inference phase, all approaches—except [19] and [12]—rely on selecting the most probable activity or on randomly selecting this activity based on the activities probability distribution in the prediction. On the other hand, the approaches that do not utilize encoder-decoder architectures are constrained to predict the same features they use as input. In this case, either they only use the activities as inputs [18], losing information that is available in the event log, or they also use additional information, which accumulates errors during the inference phase by predicting features that are not relevant to the predictive task. Finally, none of the aforementioned approaches take advantage of the information accessible in the process model. This information could facilitate suffix prediction as it provides insight into all potential execution paths a case may follow. Consequently, the neural network could focus more on suffix prediction rather than becoming overly focused on predicting the next activity, a scenario that could potentially result in a suboptimal suffix prediction.

In this paper, we present ASTON, an approach based on an encoder-decoder architecture enriched with graph recurrent neural networks [11], with an attention mechanism that helps to retain information when the input prefixes are considerably long. We argue that the strategy used to select the next activity during the suffix prediction phase is essential to obtain a good performance. In particular, we propose the usage of a beam search strategy because it brings a balance between a greedy search and exhaustively exploring the whole tree of possible suffixes. Unlike [19] and [12], we also state that length normalization during the beam search is essential to avoid predicting suffixes that are too short. Our neural model is enriched with information from the process model, which is automatically extracted using recurrent graph neural networks. We hypothesize that integrating information about the whole process model can not only assist the model in understanding the set of possible suffixes to predict but also better align the output probabilities from the model to accurately reflect the suffix prediction task. In summary, the main contributions of our approach are as follows:

- A novel encoder-decoder neural model with an attention mechanism that is trained to predict the activity suffix, instead of being trained to predict the next activity. Our neural model decouples the prediction of the suffix from the representation learning of the prefix, so it does not need to learn to predict features irrelevant to the prediction task.
- A new graph recurrent neural network approach that extracts the information from the prefix and the process model, which is used to enhance the feature extraction phase from the prefix. This information is used to initialize the decoder part of the neural network, ensuring that the information from the process model is effectively utilized.
- An improved beam search strategy that selects the most promising suffix based on a heuristic search, achieving a good balance between the approach performance and the computational time to obtain a result.
- An extensive validation of ASTON using 12 real-life event logs, comparing the results against six different approaches of the state of the art. These results show that ASTON obtains the best results and that there are statistically significant differences with the other approaches.

The remainder of this paper is structured as follows. Section 2 presents the state-of-the-art predictive monitoring techniques based on deep learning; Sect. 3 introduces formally the definitions and concepts that are necessary to understand the approach; Sect. 4 describes our approach in detail; Sect. 5 shows the evaluation of our approach using real-life event logs, and Sect. 6 highlights the conclusions of the paper and future work.

2 Related work

The first approaches that focused on predicting the activity suffix of a running case using deep learning were [17] and [18]. [17] uses an LSTM neural network to predict the next activity and the time until the next event, encoding the activities using one-hot encoding and adding multiple time features extracted from the events. Then, during the suffix prediction phase, both the time and the next activity are iteratively predicted, which penalizes the overall activity suffix prediction since errors between successive event predictions are accumulated until the end. In [18] a similar approach is followed, but it only uses the activities of the event log, which are encoded as embeddings, meaning that relevant information for the activity suffix prediction, such as time- and resource-related features, is missing. Furthermore, both approaches use either argmax sampling [17] or random sampling [18], which does not select the suffix with the highest probability overall.

[19] is built upon the architecture of [17] but it controls the selection of the next activity during the suffix prediction phase by checking whether the prediction is compliant or not with a set of LTL-mined rules. It uses a beam search to generate the suffix prediction. [20] is also built upon the work of [17], but it focuses on recommending the best action—most appropriate activity according to a key process indicator, KPI—for a running case, predicting the suffix as a support for their system. It trains a multitask LSTM network to predict both the next activity and the KPI for a given event. Then, it predicts the suffix through the neural network and selects the most similar suffix that is compliant with a process model and maximizes the target KPI. Using that suffix, the next best action is recommended. These approaches suffer from the same drawbacks as [17]. Furthermore, [19] does not normalize the beam search so this approach tends to predict suffixes that are shorter than the ground truth ones.

Other approaches focus on finding novel ways to encode the information from the event log. [21] clusters the resources into roles and then learns embeddings for both activities and roles. It iteratively predicts each activity of the suffix from a prefix by using an LSTM that is trained to predict both the next activity as well as the role that performs the activity and the time to the next event. [22] encodes the whole prefix into a single representation through embeddings as well as some temporal information such as the time between events, the hour and the day of the month for each event, etc. Then, a feed-forward network is trained to predict the next suffix identifier, hence achieving a one-shot suffix prediction. [23] follows a similar approach to [20]. First, it replays the prefixes on a Petri net in order to obtain the set of activated places—behavioral context—; then, an LSTM is trained to predict the next sequence of attributes given a prefix of attributes—data context—; and, finally, given a certain prefix during inference, the most similar suffix from the training set is selected according to both its behavioral context and data context. These approaches still suffer the same drawbacks as [17], which are even aggravated since they use more information than [17], so more errors are accumulated in the inference phase. Furthermore, in [22] the encoding step of both whole prefixes and suffixes into a single embedding is prone to lose a lot of information, so the predictions

are bound to be inaccurate. [25] proposes to split the prefixes into buckets and then search for a set of input attributes of the event log whose prediction probability is consistent across every set of attributes—invariant set of attributes—. This search procedure is done by learning an LSTM with a technique called Invariant Risk Minimization [27].

Some approaches are based on novel architectures to deal with the suffix prediction problem. [12] presents an encoder-decoder LSTM that is trained as a Generative Adversarial Network—GAN—. Specifically, the encoder-decoder acts as a generator of suffixes that aims to fool the discriminator, which is also an LSTM trained to distinguish between fake suffixes from the generator and real suffixes. [14] compares multiple architectures for suffix prediction such as LSTMs, recurrent autoencoders—either trained by maximum likelihood estimation or by using a GAN approach—, or transformers [28], concluding that no architecture clearly outperforms the other ones in a wide variety of event logs. [24] trains an LSTM to predict the next event, time, and attributes in a many-to-many fashion, using what authors call a *modulator*, which learns the importance of each attribute in the prediction of the event. [15] trains an encoder-decoder to predict the next activity based on two LSTMs coupled with an attention mechanism. Then, to predict the suffix, this model is used iteratively by always selecting the most probable activity. In [12] and [14]—the transformer variant—the decoder is also trained to predict the same features that are used in the encoder, so it still accumulates errors from predicting features that are not relevant for the predictive task. While [24] still has the aforementioned drawback, [15] avoids it by training an encoder-decoder to predict the next activity. However, the decoder only has information about the next activity during training and it is not trained specifically to predict the entire suffix. [16] adapts the Differentiable Neural Computer to predictive monitoring by separating the encoding and decoding phase of the neural model and preventing the decoder to write into the memory during the decoding phase. [26] trains the GAN-LSTM proposed by [29] for the next activity prediction problem by using reinforcement learning so that the predictions are compliant with a process model and take into account an estimated value of a KPI for that prediction.

Finally, it is worth noting that time series analysis, commonly applied in forecasting numeric data sequences, also offers methodologies that could be adapted for predictive monitoring. Although typical time series models primarily handle numeric data and might not directly address the categorical and complex nature of event logs in business processes, their analytical techniques could provide additional insights, particularly in understanding temporal patterns and forecasting event timings within process sequences. However, without significant adaptations, these models often fail to adequately capture the intricate interdependencies and logical or causal relationships critical in business process contexts, which rely not only on temporal proximity but also on the unique rules governing the processes. Furthermore, approaches that leverage Graph Neural Networks, such as [11, 30], are focused on predicting only the next activity. These models do not address the challenge of predicting the full sequence of activities, thereby overlooking the nuances of suffix prediction.

To overcome the shortcomings of existing state-of-the-art approaches, we propose an encoder-decoder architecture, enriched with an additive attention mechanism,

designed to predict the sequence of activities given a prefix of events. The encoder module ingests the process model's structural data and assimilates information from the activities, timestamps, and resources detailed within the event log. On the other hand, the decoder exploits the features learned by the encoder to exclusively forecast the activity suffix. The attention mechanism acts as a nexus between these two modules, assessing the pertinence of the prefix's learned context for each subsequent prediction generated by the decoder. During the training phase, the decoder applies teacher forcing, accelerating the convergence process. In this stage, it harnesses information from the complete suffix to predict the outcome. However, in the inference phase, a normalized beam search is deployed to mitigate the exposure bias inherent in training with teacher forcing.

3 Preliminaries

This section presents some basic concepts needed to understand our approach and the basic building blocks that will be used to construct our neural architecture to predict the activity suffix of a running case.

Definition 1 (event) Let A be the domain of activities, Z the domain of case identifiers, T the domain of timestamps, and R the domain of resources. An event, $e \in E$, is a tuple (a, z, t, r) , where $a \in A$, $z \in Z$, $t \in T$, and $r \in R$.

Definition 2 (projection functions) Let $e = (a, z, t, r)$ be an event. Then its projection functions in its activity, case identifier, and timestamp are denoted, respectively, by π_A , π_Z , π_T , and π_R , i.e., $\pi_A(e) = a$, $\pi_Z(e) = z$, $\pi_T(e) = t$, and $\pi_R(e) = r$.

An ordered sequence of events that belongs to the same case is denoted as a trace.

Definition 3 (trace) Let S be the universe of traces. A trace $\sigma \in S$ is a non-empty sequence of events $\sigma = \langle e_1, \dots, e_n \rangle$, such that for all $e_i, e_j \in \sigma$, with $i, j \in [1, n]$; if $j > i$, then $\pi_C(e_i) = \pi_C(e_j)$ and $\pi_T(e_j) \geq \pi_T(e_i)$. Here, $|\sigma| = n$.

Table 1 Potential example of an event log from a hospital

Case ID	Activity	Timestamp	Resource
Case1934	Registration	15-02-2021 08:34:20	Max
Case1934	CT scan	16-02-2021 12:35:12	Lewis
Case1934	Consultation	20-03-2021 11:23:53	Nicholas
Case1934	Blood analysis	21-03-2021 10:54:12	Sergio
Case2032	Registration	17-02-2021 12:57:25	Charles
Case2032	Blood analysis	17-02-2021 13:05:24	Carlos
Case2032	Discharge	17-02-2021 18:46:26	Fernando

Given the previous definition of trace, if two events have exactly the same timestamp, they are ordered following the already found order in the event log.

Definition 4 (event log) An event log is a subset of the universe of traces, i.e., $L = \{\sigma_1, \dots, \sigma_l\} \subseteq S$, where $|L| = l$, is the number of traces in the event log.

To illustrate these concepts, let Table 1 serve as an example of an event log that records the steps performed in the process of patient management in a hospital. In this event log, there are five different activities, each one representing a step in the process. For each event —row in the table—, the case identifier, activity, timestamp, and the resource assigned to its execution, are recorded. The sequence of events with the same case identifier is called a trace. In this example, there are two traces, with case identifiers *Case1934* and *Case2032*. The events for each trace appear chronologically ordered according to their timestamp. Note that the only mandatory elements in an event log are the case identifiers, the activities, and a partial order over them within each trace.

Predictive monitoring approaches work by using prefixes of events as their inputs.

Definition 5 (event prefix and suffix)

Let $\sigma \in S$ be a trace from the universe of traces S such that $\sigma = \langle e_1, \dots, e_n, e_{end} \rangle$ where e_{end} is a special “end of case” token, and $k \in 1, \dots, n$. Then, the k -event prefix is defined as $hd^k(\sigma) := \langle e_1, \dots, e_k \rangle$. The k -event suffix is defined as $tl^k(\sigma) := \langle e_{k+1}, \dots, e_n, e_{end} \rangle$.

The activity, timestamp, and resource of both the event prefix and suffix can be obtained by applying the projection functions π_A , π_T , and π_R to $hd^k(\sigma)$ and $tl^k(\sigma)$, respectively. Thus, let us first define the next activity prediction problem, which can be dealt with as the prediction of the first suffix activity, $hd^1(tl^k(\sigma))$:

Definition 6 (next activity prediction problem)

Let $hd^k(\sigma) = \langle e_1, \dots, e_k \rangle$ be a prefix of length k of a trace $\sigma \in S$ from the universe of traces S , and let $hd^1(tl^k(\sigma)) = \langle e_{k+1} \rangle$ be the first event of the suffix of σ after the k -th event. The aim of the next activity prediction problem is to find a function Ω_A that satisfies $\Omega_A(hd^k(\sigma)) = \pi_A(e_{k+1})$ for all $\sigma \in S$ and k in the range of valid prefix lengths, where π_A is a projection function for the activities.

Accordingly, the suffix prediction problem can be defined as a generalization of the next activity prediction problem, where the objective is to predict the k activities of the whole suffix, $hd^k(tl^k(\sigma))$, not only the first suffix activity:

Definition 7 (activity suffix prediction problem)

Let $hd^k(\sigma)$ be a prefix of length k of a trace $\sigma \in S$ from the universe of traces S . The aim of the activity suffix prediction problem is to find a function Ω_{AS} such that $\Omega_{AS}(hd^k(\sigma)) = \pi_A(tl^k(\sigma))$ holds for all $\sigma \in S$ and for all valid k .

However, the general problem of the activity suffix prediction problem can be approached by iteratively predicting the next activity until the end of the trace is found. Formally, this type of prediction can be defined as follows:

Definition 8 (iterative activity suffix prediction) Let $\oplus$ be the concatenation operator. Then, the function Ω_{AS} can be constructed by iteratively predicting the next activity using Ω_A following the recursive definition:

- $\Omega_{AS}(\text{hd}^k(\sigma)) = \sigma$ if $\Omega_A(\text{hd}^k(\sigma)) = e_{\text{end}}$.
- $\Omega_{AS}(\text{hd}^{k+1}(\sigma \oplus e)) = \Omega_{AS}(\text{hd}^k(\sigma)) \oplus a$, with $\pi_A(a) = \Omega_A(\text{hd}^k(\sigma))$, otherwise.

Our approach extracts structural information from the process model to bolster the learned representation of the event prefix. The process model is represented as a Petri net, which can be defined as follows:

Definition 9 (Petri net) A Petri net is a tuple $N = (\mathcal{P}, \mathcal{T}, \mathcal{F}, \mathcal{M}_i)$, where $\mathcal{P}$ is a set of places, $\mathcal{T}$ is a set of transitions, $\mathcal{F} \subseteq (\mathcal{P} \times \mathcal{T}) \cup (\mathcal{T} \times \mathcal{P})$ is a set of arcs, and $\mathcal{M}_i : \mathcal{P} \rightarrow \mathbb{N}$ is the initial marking.

Given a prefix of activities and a Petri net, it is possible to obtain the *token replay* of the prefix in the model, which is the sequence of transitions and token movements within the places that would result from reproducing the behavior of the prefix in the model. On the other hand a Petri net can be converted into a *place graph*, which is comparatively smaller relative to a Petri net. A place graph can be defined as follows:

Definition 10 (place graph) A place graph is a directed graph $G = (\mathcal{P}, \mathcal{E})$ obtained from a Petri net $N = (\mathcal{P}, \mathcal{T}, \mathcal{F}, \mathcal{M}_i)$. Here, $\mathcal{P}$ is a set of places and $\mathcal{E} \subseteq \mathcal{P} \times \mathcal{P}$ is a set of edges such that every edge connects two places $p_1, p_2 \in \mathcal{P}$. This connection is valid if p_1 and p_2 are interconnected by a transition $t \in \mathcal{T}$ in the Petri net, denoted as $(p_1, p_2) \in \mathcal{E} \rightarrow (p_1, t) \in \mathcal{F} \wedge (t, p_2) \in \mathcal{F}$.

Both the petri net and the place graph play a role in extracting the spatio-temporal interactions of the activity prefix with the process model. The spatial information of the process model is represented as the adjacency matrix of the place graph, while the temporal information is encapsulated as the replay of the prefix in the Petri net.

The information of event replay in the petri net is combined with the adjacency matrix using a *graph convolutional network* [31] (GCN), which is formally defined as follows.

Definition 11 (graph convolutional network) A graph convolutional neural network operates on a feature matrix X and an adjacency matrix with self-loops $\tilde{\mathcal{A}} = \mathcal{A} + I_N$, defined as: $GCN(\mathcal{A}, X) = \tilde{D}^{-1} \tilde{\mathcal{A}} X \Lambda$, where $\tilde{D}_{ii} = \sum_j \tilde{\mathcal{A}}_{ij}$ is the degree matrix of $\tilde{\mathcal{A}}$ and Λ is a trainable weight matrix.

The inclusion of self-loops is crucial for the graph convolutional network because it enhances its convergence and avoids the problem of exploding/vanishing gradients. Moreover, the expression $\tilde{D}^{-1}\tilde{A}$ is preferred over the normalization $\tilde{D}^{1/2}\tilde{A}\tilde{D}^{1/2}$ because it takes into account the directionality of the graph structure, an important aspect when the adjacency matrix represents a Petri net [11].

While the graph convolutional network is enough to extract the spatial information from a single event, it falls short in extracting the temporal information from the full prefix. To address this, we employ a graph recurrent neural network (GRNN) [11, 32] to simultaneously extract both temporal and spatial information from the activity prefix. A GRNN incorporates the graph neural network operation within the recurrent neural network operations. In this paper, we use GRNNs based on GRUs, owing to their computational efficiency over LSTMs while providing similar performance, as expressed in Eq. 1.

$$\begin{aligned} z_t &= \sigma(GCN(x_t, \mathcal{A}) + GCN(h_{t-1}, \mathcal{A}) + b_z) \\ r_t &= \sigma(GCN(x_t, \mathcal{A}) + GCN(h_{t-1}, \mathcal{A}) + b_r) \\ \tilde{h}_t &= \tanh(GCN(x_t, \mathcal{A}) + r_t \circ GCN(h_{t-1}, \mathcal{A}) + b_z) \\ h_t &= z_t \circ h_{t-1} + (1 - z_t) \circ \tilde{h}_t \end{aligned} \quad (1)$$

In the previous, equation 1, $\mathcal{A}$ is the adjacency matrix of the place graph, x_t is the feature matrix built after replaying the event e_t on the Petri net, and h_t is the hidden state of the GRNN.

4 Deep learning architecture description

4.1 ASTON architecture for training

In this section, we present our approach, ASTON (Activity Suffix predicTiOn based on eNcoder-decoder), and approach for predicting activity suffixes given input event prefixes. ASTON leverages an encoder-decoder recurrent neural model equipped with an attention mechanism, as illustrated in 1. ASTON is built upon three key components:

- An encoder, which learns an effective representation of the input prefix and its interactions with the process model.
- A decoder, which predicts the suffix using the information obtained from the encoder.
- An attention mechanism, which computes the relevance of reach decoder hidden state to the input prefix, aiding prediction accuracy.

Diving into the specifics of each module, the encoder consists of two distinct parts. The first is a Gated Recurrent Unit (GRU) [33] that learns from the input event prefix, which contains information about activities — a_i in Fig. 1 — as well as the time and resources for each event, e_i . The second part is a Graph Recurrent Neural Network, which is trained using the replay of the activity prefix within the

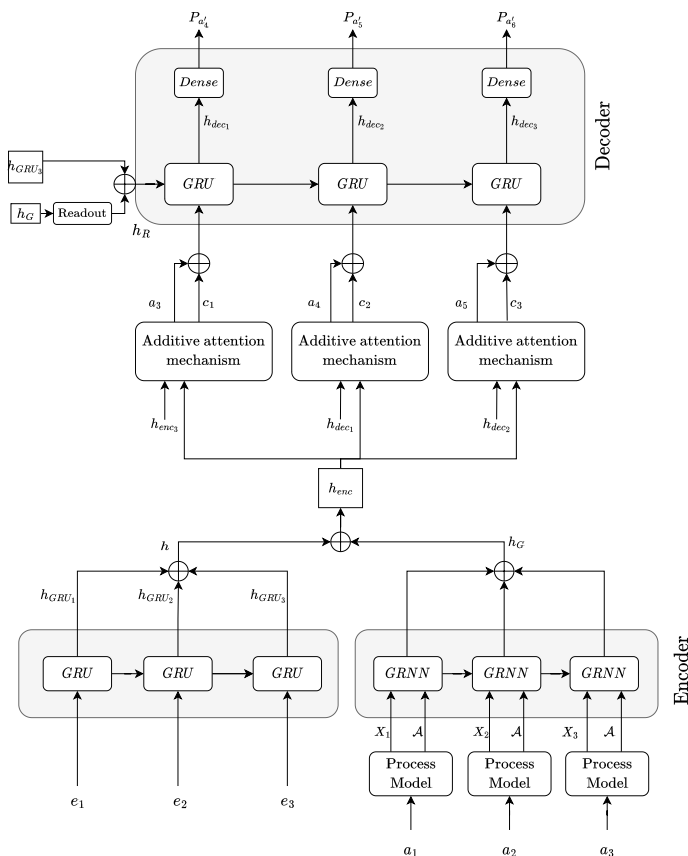


Fig. 1 ASTON architecture for the training phase

process model. The output of the decoder is a sequence of hidden states, h_{enc} , each representing an event in the prefix.

This sequence of hidden states serves two main purposes. Firstly, it generates a tensor which is used to initialize the hidden state of the decoder. This tensor comprises two different parts: one half derived from the GRU's final hidden state, h_{GRU_3} , while the other half derived from a readout operation that summarizes all the hidden states produced by the GRNN along the prefix. This design ensures that the decoder has direct access to both the event log information and the process model. Notably, the readout operation is performed over the full sequence of hidden states, h_G , instead of the last hidden state, as in [11], so that the information since the beginning of the prefix is better propagated towards the summarized tensor. Formally, this readout operation is performed by applying the max-pooling operation over the length of the prefix (k) and the number of places of the Petri net $|\mathcal{P}|$. If $h_G \in \mathbb{R}^{k \times |\mathcal{P}| \times H_G}$ is the sequence of hidden states, where, F is

the number of features, and H_G is the number of hidden features learned by the GRNN, then the result of the readout operation, $h_R \in \mathbb{R}^{H_G}$, is:

$$h_R = \max_{(1 \leq i \leq k, 1 \leq j \leq |\mathcal{P}|)} X^{(i,j,h)} \quad \forall h \in H_G \quad (2)$$

Secondly, it provides the sequence of hidden states, h_{enc} , utilized by the decoder and the attention mechanism for the activity suffix prediction task. This sequence, akin to the tensor used to initialize the decoder, has half-part information from the event log and half-part information from the process model. Therefore, the attention mechanism will also have direct access to the information of the process model. Also, the information used to initialize the hidden state of the decoder will be balanced with respect to the information it gets from the context vector created by the attention mechanism.

The decoder, implemented as a GRU, outputs a sequence of hidden states, h_{dec_n} , which are subsequently projected into the probability distribution of the next activity of the suffix, $P'a_n$. The input at each timestep is a concatenation of the context vector (computed using the attention mechanism) and the preceding activity. To speed up the convergence and training of the predictive model, *teacher forcing* [34] is employed, where each ground-truth activity a_i is used at each timestep instead of the predictions a'_i of the decoder. To predict the first event of the suffix, the decoder is fed with the last event of the prefix — a_3 in Fig. 1—.

The attention mechanism then computes a context vector c_i that represents the relevance of each hidden state of the decoder, h_{dec_n} , with respect to the full sequence of hidden states of the encoder, h_{enc_n} . For the first event of the decoder, as there are no prior decoder hidden states available, the last state of the encoder is used — h_{enc_3} in Fig. 1—. Most notably, since the GRNN outputs are directly feeded to the decoder, the attention mechanism has direct access to the information from the process model, instead of being diluted if we also used the GRU to preprocess it. In this paper, we employ an additive attention mechanism [35], the specifics of which will be further elaborated in subsequent sections.

4.1.1 Input encoding

ASTON encodes each event as a tensor resulting from the concatenation of the activity, with the time features computed from the timestamps, and the resources assigned to each activity. Activities are first encoded into integer identifiers and then into embeddings, represented by a vector assigned to each activity. Embeddings are used because they are less memory intensive than a one-hot encoding representation in event logs with a high number of activities or resources.

On the other hand, the timestamps of the event cannot be used directly as inputs to the neural network, so the following time features are computed from these timestamps: the time elapsed between one event and its predecessor; the time elapsed between one event and the beginning of the trace; the time elapsed between one event and the midnight of the same day; and the month, the day of the week, and the hour of the day in which the event has happened. The time elapsed between one event and its predecessor and the time elapsed between one event and the start of the

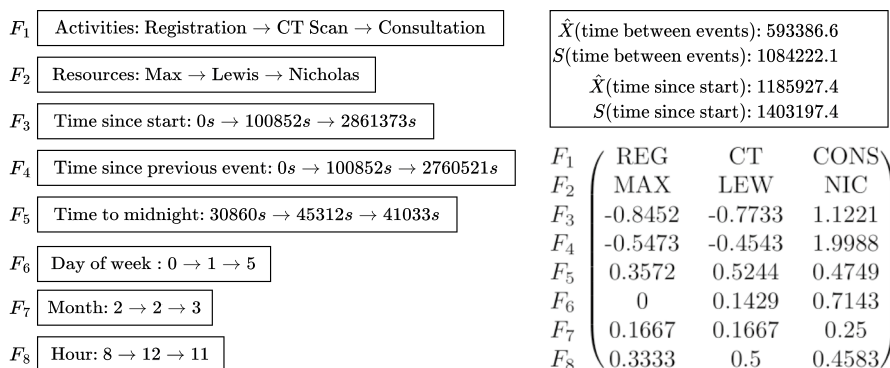
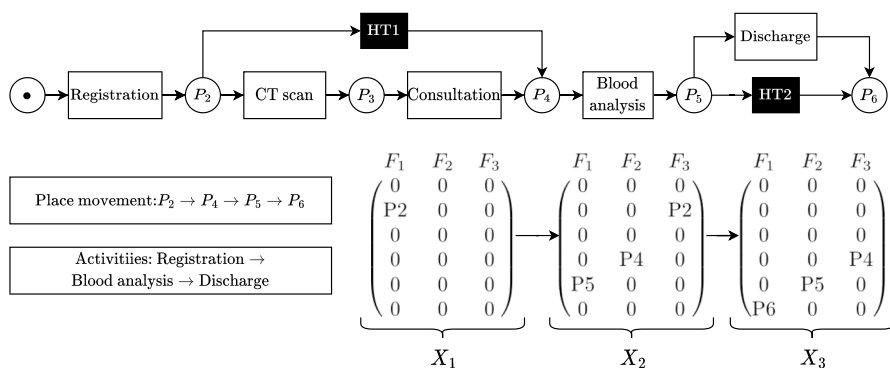


Fig. 2 Encoding of the prefix corresponding to the three first events of the trace *Case1934* from the log shown in Table 1

trace are standardized using the z-score technique [36] while the other time-related variables are normalized using the maximum value for each type of variable.

Figure 2 shows an example of the tensor that encodes the information of a prefix. In this tensor, columns represent the prefix events, while rows represent the features computed for each prefix event. Thus, for a given column, i.e., for an event, the feature F_1 refers to the activity; F_2 indicates the resource; and F_3, F_4, F_5, F_6, F_7 , and F_8 are the normalized time-related features calculated from the event timestamp.

In terms of the inputs to the GRNN module, we construct an input feature matrix of dimensions $\mathbb{R}^{L \times |\mathcal{P}| \times k}$, where L stands for the length of the prefix, $|\mathcal{P}|$ is the number of places in the Petri net process model, and k represents the most place movements in the Petri net. Each of these latter movements is represented in a separate column of the feature matrix, with each column corresponding to an individual place. Therefore, by recording each place activation individually in each column, the GCN by itself can explicitly account for the temporal aspect of token movement within the



Petri net, alleviating the computation from the RNN part of the GRNN and easing the propagation of the information along the prefix.

Let Fig. 3 be an example of this encoding. This figure displays the feature matrix generated after replaying the trace identified as “Case2032”, using $n = 3$. The feature matrix corresponding to each event in the trace forms a sequence, denoted as X_1, X_2, X_3 , which would serve as the input to the GRNN. It is noteworthy that even if the activation of P_2 is absent from X_3 , this information is still propagated through the prefix. This is because the GRNN processes the entire prefix, rather than just the last activations. In our experiments, we empirically found that setting $n = 5$ offers a good balance, performing well without significantly impacting computational efficiency or neural model complexity.

4.1.2 Attention mechanism

ASTON is an architecture that uses an attention mechanism to connect its encoder and decoder module because relying solely on the last hidden state of the encoder may not yield accurate predictions, as it assumes this state fully encapsulates the execution context of the prefix. To overcome this limitation, an attention mechanism is used to leverage all information from the encoder. This attention mechanism computes a context vector, c_t , that summarizes the information of the encoder relevant to the current state of the decoder through a three-step process:

- **Attention score calculation:** ASTON calculates attention scores by adding the encoder’s hidden states to the decoder’s current hidden state, as per the method described by Bahdanau et al. [35]. This scoring is mathematically expressed as $s_{t,i} = w^T \tanh(W_1 h_{dec_{t-1}} + W_2 h_{enc_i})$, where W_1 , W_2 , and w^T are learnable parameters, $h_{dec_{t-1}}$ is the decoder’s previous hidden state, h_{enc_i} is the encoder’s i_{th} hidden state, and $s_{t,i}$ is the resulting score.
- **Attention distribution:** The softmax function is applied to these scores to determine the model’s focus on parts of the input data, using $\alpha_{t,i} = \frac{\exp(s_{t,i})}{\sum_{j=1}^E \exp(s_{t,j})}$, where E denotes the sequence of hidden encoder states.
- **Context vector generation:** The context vector c_t is then generated by weighting the hidden states of the decoder with these attention scores, following $c_t = \sum_{i=1}^E \alpha_{t,i} h_{enc_i}$.

The context vector, c_t , combined with the previously predicted activity, a'_{t-1} (or the last activity of the prefix for $t = 0$), using the concatenation operator $\oplus$, forms the input I_t for the decoder at each timestep t . This detailed process enables ASTON to focus on the relevant parts of the input sequence for each decoding step, enhancing the prediction’s accuracy by incorporating contextually significant information.

Figure 4 visually outlines the process of creating the context vector, c_t , as used in the attention mechanism in ASTON. This process involves the hidden

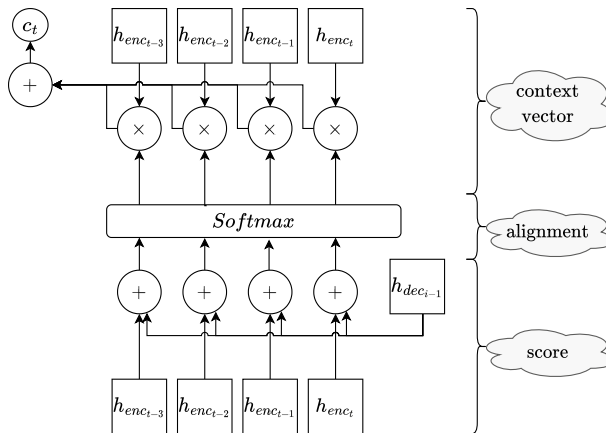


Fig. 4 Attention mechanism of ASTON when processing an input prefix. The circles with a + inside represent the addition operation, and the circles with a $\times$ inside represent the product between two vectors

states of the encoder ($h_{enc_{t-3}}, \dots, h_{enc_t}$), the last state of the decoder ($h_{dec_{t-1}}$), and the context vector (c_t) itself. The lower part of the picture depicts the first state of the attention mechanism, where scores are computed by adding the hidden states of the encoder and the last state of the decoder in a pairwise manner. The middle part of the figure then demonstrates how these calculated scores are aligned by applying a softmax operation. Lastly, the top part of the figure shows the production of the final context vector, c_t , achieved by multiplying each of the hidden states of the encoder with the aligned scores and summing the results.

4.1.3 Training

As the activity suffix prediction can be viewed as a multiclass prediction problem of a sequence of activities, ASTON is trained using the cross-entropy loss function defined for activities. Thus, ASTON applies the cross-entropy loss to each predicted activity of the suffix. Formally, let a be a ground-truth activity index, $P_{a'}$ represent the predicted probability, and $|A|$ be the number of different activities in the event log. Then, the cross-entropy loss for activities is defined as follows:

$$l(a, a') = -\log \left(\frac{\exp(P_{a'})}{\sum_{i=1}^{|A|} \exp(P_i)} \right) \quad (3)$$

Then, this loss function is extended to the whole predicted suffix as follows:

$$L(\text{tl}^k(\pi_A(\sigma)), \text{tl}^k(P_{\pi_A(\sigma')})) = \sum_{k=1}^K l(a_k, P_{a'_k}) \quad (4)$$

where $\text{tl}^k(\pi_A(\sigma))$ is the ground-truth activity suffix, $\text{tl}^k(P_{\pi_A(\sigma')})$ is the sequence of probability distributions of the suffix, K is the length of the suffix, a_k is the activity index k , and $P_{a'_k}$ is the probability distribution of the predicted activity k . Note that both $\text{tl}^k(\pi_A(\sigma))$ and $\text{tl}^k(P_{\pi_A(\sigma')})$ are padded with end-of-case tokens to the maximum length of the log trace, so they are guaranteed to have the same length during training.

4.2 ASTON architecture for inference

In the inference phase of ASTON, the decoder cannot use the ground truth activities as inputs, unlike during the training phase, as Fig. 5 shows. Instead, it substitutes these with the predicted activities, $\text{tl}^k(\pi_A(\sigma'))$, except for the first activity, which is the last activity of the input prefix, which is also known during inference. This introduces an *exposure bias*, stemming from the difference between the training inputs (ground truth) and inference inputs (predicted outputs). This makes the selection of the next activity of the suffix during inference critically important.

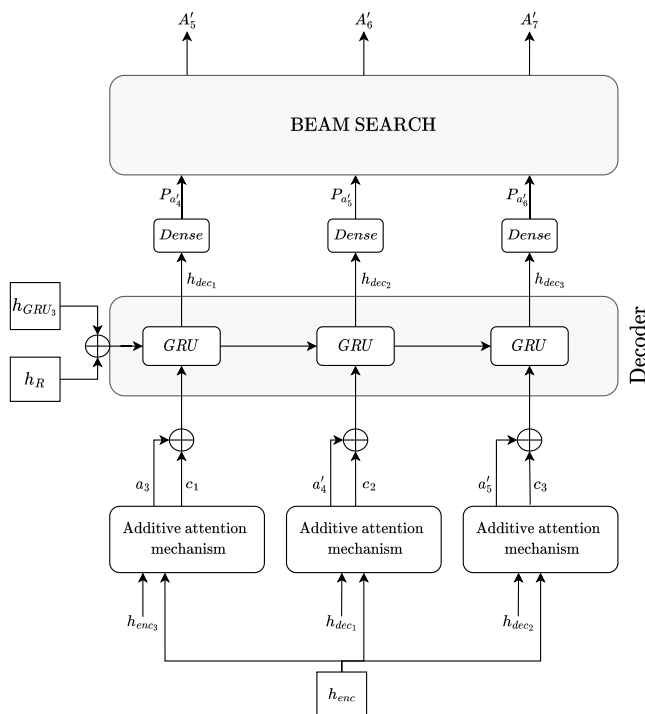


Fig. 5 ASTON architecture for the inference phase (encoder omitted for clarity)

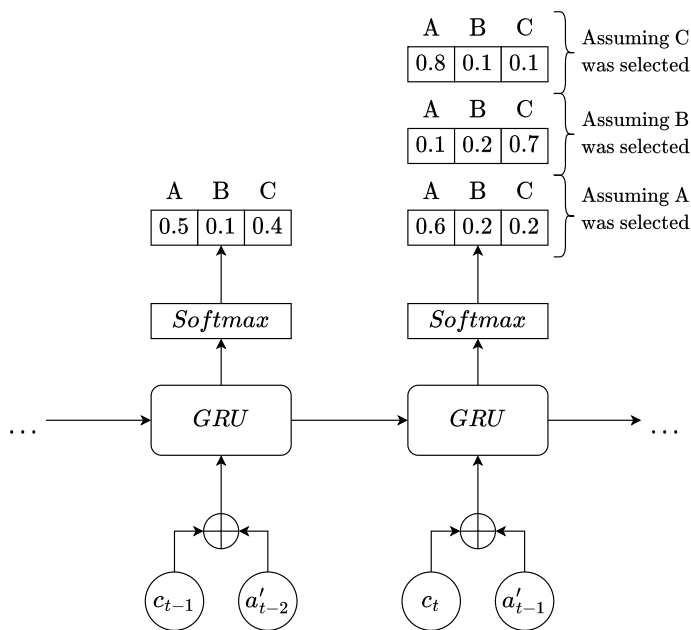


Fig. 6 Output probabilities for the next two activities given a prefix, where the numbers represent the probability of each activity at a timestep

Each step in the sequence outputs a probability distribution over the set of activities, $\text{tl}^k(P_{\pi_A(\sigma')})$, from the decoder, from which the predicted activity a'_t must be selected. Figure 6 demonstrates the possible output probabilities that would be generated by the decoder after predicting the next two activities for a given prefix. In predictive process monitoring, two strategies for activity selection are commonly used:

- *Argmax* [17, 21], always selects the most likely activity at each prediction step. For instance, if we look at Fig. 6, the activities “A” and “A” would be selected, producing the suffix “AA” with a joint probability of $0.5 \cdot 0.6 = 0.3$. This strategy, as noted in [37], works reliably in event logs with short traces while underperforming in logs with long ones.
- *Random sampling* [18, 21], where the next activity is randomly sampled from the probability distribution of the next activities. Returning to the example in Fig. 6, we might select “B” as the next activity, followed by “A” as the final prediction, yielding the predicted suffix “BA” with a joint conditional probability of $0.1 \cdot 0.8 = 0.08$. This strategy, unlike *argmax*, performs better in logs with long traces while underperforming in logs with short ones, as noted in [37].

However, both strategies might not select the suffix with the highest overall probability (“AC”). The *argmax* strategy lacks exploration and can select a suboptimal solution or even repeatedly select the same set of activities, never stopping. On the

contrary, the *random sampling* strategy can predict suboptimal suffixes due to its overly broad exploration. Both strategies only consider the probabilities of the current prediction and not the probabilities of the whole suffix.

To address these limitations, ASTON employs the beam search [38], an optimization of exhaustive search that significantly reduces both memory and time requirements while maintaining good solution quality. The time complexity of the beam search algorithm is $\ll O(B_w \cdot |A| \cdot k)$, where B_w is the beam width (representing the number of alternative suffixes kept at each decision point), $|A|$ is the number of different activities in the log, and k is the maximum length of the suffix. Thus, the beam search operates by maintaining only the B_w most promising sequences at each decision point and discarding the rest. This contrasts with the exhaustive search, which would keep all possibilities open, quickly leading to an infeasible number of alternatives as the length of the sequence increases.

As an example, let us refer to Fig. 7, which demonstrates the operation of beam search given the output probabilities shown in Fig. 6. For a beam width of $B_w = 2$, only the two most probable alternatives are kept at each step. In the first step, the activities “A” and “C” are chosen for further exploration since they have the two highest probabilities (0.5 and 0.4, respectively); activity “B” is discarded as it has the lowest probability (0.1). The alternatives are expanded in the order of their probability; in this case “A” is expanded before “C”. In the second step, the search stops as we have reached the end of the suffix prediction. The suffix with the highest joint-conditioned probability is then selected from the remaining activities. In this example, that would be the suffix “AC”. In particular, the joint-conditioned probability of a partial suffix is formally computed during the beam search through the following expression:

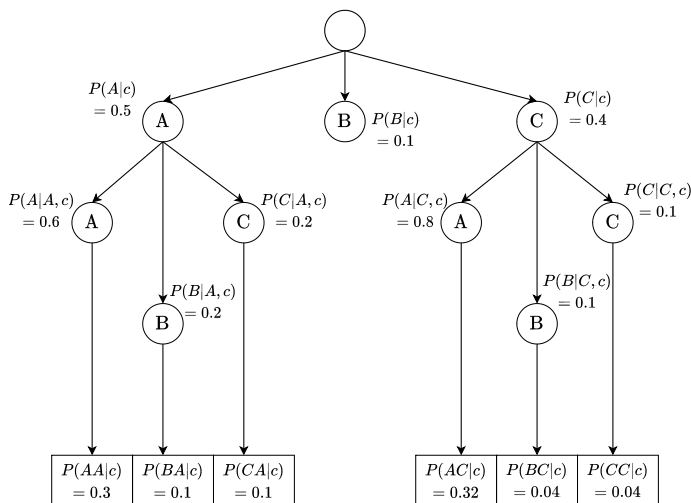


Fig. 7 Representation of the probability tree of Fig. 6

$$\log P(a'_i, \dots, a'_0 | \text{hd}^k(\sigma)) = \sum_{j=0}^i \log P(a'_j | a'_{j-i}, \dots, a'_0, \text{hd}^k(\sigma)) \quad (5)$$

where $\text{hd}^k(\sigma)$ is the prefix that conditions the prediction; a'_i is the i_{th} activity, with $i \in \{0, \dots, N-1\}$; and the logarithm operation is applied to deal with the numerical instability of consecutive products of floating-point numbers in the interval $[0, 1]$.

However, the expression 5 tends to favor short suffixes since a negative log probability is added for each predicted event, yielding a lower score for the predicted suffix [39]. Therefore, we have adopted the same normalization strategy as in [39] to fairly compare suffixes of different lengths, resulting in the following expression:

$$\log P(a'_i, \dots, a'_0 | \text{hd}^k(\sigma)) = \frac{(5+1)^\alpha}{(5+i)^\gamma} \sum_{j=0}^i \log P(a'_j | a'_{j-i}, \dots, a'_0, \text{hd}^k(\sigma)) \quad (6)$$

where γ is a factor which is set to 0.65 in our experimentation. This suffix normalization is what distinguishes the beam search implemented in ASTON from the beam search proposed by [19] and [12]. Thus, [19] does not perform any kind of normalization to the output probabilities, whereas [12], instead of normalizing, stops the prediction if the suffix length exceeds a certain threshold related to the average trace length of the log. Therefore, their stopping strategy is too restrictive by not allowing the prediction of suffixes longer than this threshold.

5 Evaluation

5.1 Experimental setup

In this study, ASTON was evaluated against five other state-of-the-art predictive monitoring approaches under identical experimental setups, as detailed in [37] which focus on predicting the next activity suffix. We discarded the approaches that focus only in predicting the next activity and do not provide any mean to predict the activity suffix. Among the former approaches, the method by [21] was also tested using two different sampling strategies. The same set of hyperparameters was employed across all event logs for ASTON: training for 150 epochs with a batch size of 64, 32 units for both the embedding dimension and all the hidden units, the Adam optimizer with a learning rate of 0.005, and a beam width of 5. This uniform hyperparameter strategy underscores ASTON's robustness, contrasting with the approach by [21], which underwent random search optimization as reported. The other approaches used the same hyperparameter settings specified in their respective publications. Our evaluation method involves shuffling of traces before splitting them using a 5-fold cross-validation. Then, each training fold is split in a 80 and a validation set, with the validation set used for selecting the best-performing model based on its validation loss.

Table 2 Statistics of the event logs used in the evaluation

Event log	Traces	Activities	Events	Avg. case length	Max. case length	Avg. event duration	Max. event duration	Avg. case duration	Max. case duration	Variants
BPI 2012	13,087	36	262,200	20.04	175	0.45	102.85	8.62	137.22	4366
BPI 2012 A	13,087	10	60,849	4.65	8	2.21	89.55	8.08	91.46	17
BPI 2012 C	13,087	23	164,506	12.57	96	0.74	30.92	8.61	91.46	4336
BPI 2012 O	5015	7	31,244	6.23	30	3.28	69.93	17.18	89.55	168
BPI 2012 W	9658	19	170,107	17.61	156	0.7	102.85	11.69	137.22	2621
BPI 2012 W C	9658	6	72,413	7.5	74	1.75	30.92	11.4	91.04	2263
BPI 2013 C. P	1487	7	6660	4.48	35	51.42	2254.84	178.88	2254.85	327
BPI 2013 I	7554	13	65,533	8.68	123	1.57	722.25	12.08	771.35	2278
Env. permit	1434	27	8577	5.98	25	1.09	268.97	5.41	275.84	116
Helpdesk	4580	14	21,348	4.66	15	11.16	59.92	40.86	59.99	226
Nasa	2566	94	73,638	28.7	50	0.00	0.00	0.00	0.00	2513
Sepsis	1049	16	15,214	14.48	185	2.11	417.26	28.48	422.32	845

Time-related measures are shown in days

The state-of-the-art approaches have been evaluated on the same event logs as considered in [37] but, to provide a fair comparison, no results were reported for the approach [21] in the event logs *Nasa* and *Sepsis* because they do not contain information about resources and, therefore, their approach cannot be executed. Table 2 shows some relevant statistics for the event logs used in the experimentation. It is remarkable to note the high variability among the trace length since it increases the uncertainty in predicting the end of the trace for a given prefix. Moreover, there is also a high variability in the number of trace variants, which results in more difficulties during the training phase of the neural network for identifying common patterns between executions.

To compare the results of the state-of-the-art approaches, the same evaluation metric as in [37] has been used: the Damerau-Levenshtein normalized distance [40]. For the sake of brevity, let $q = t^i(\pi_A(\sigma))$, be a ground truth activity suffix of length i , and $q' = t^j(\pi_A(\sigma'))$ a predicted suffix of length j . Then, the Damerau-Levenshtein distance, $lev_{q,q'}(q, q')$, computes the edit distance between the ground truth suffix, q and, the predicted suffix, q' , i.e., how many substitutions, transpositions, deletions, and insertions are needed to transform the ground truth suffix into the predicted suffix. However, since suffixes are of variable lengths (we remove the trailing end of case tokens), it is more convenient to normalize this metric by the maximum suffix length, obtaining the Damerau-Levenshtein normalized distance as follows:

$$DL_norm(q, q') = 1 - \frac{lev_{q,q'}(q, q')}{\max(|q|, |q'|)} \quad (7)$$

Finally, to quantify the differences among the state-of-the-art approaches and evaluate whether the results they obtain are statistically different, a two-stage statistical comparison has been applied. Following the procedure defined in [37] and [11], Bayesian statistical tests instead of null hypothesis statistical tests were applied due to their easiness of interpretation and robustness in quantifying the differences among the approaches. In the first stage of the statistical test pipeline, a Bayesian approach to rank the approaches using the Plackett-Luce model [41] is applied, allowing the classification of multiple approaches based on their probability of obtaining the best ranking. Then, in the second stage of the statistical test pipeline, a Bayesian hierarchical test [42] between the three best approaches according to the Plackett-Luce model is performed. This second stage is especially insightful because, unlike the Plackett-Luce model, it uses the results obtained for each fold of the cross-validation evaluation procedure, so it considers that an approach can perform badly on one fold but very well on the rest, and vice versa. In order to perform these statistical tests, the library `scmamp` [43] is used.

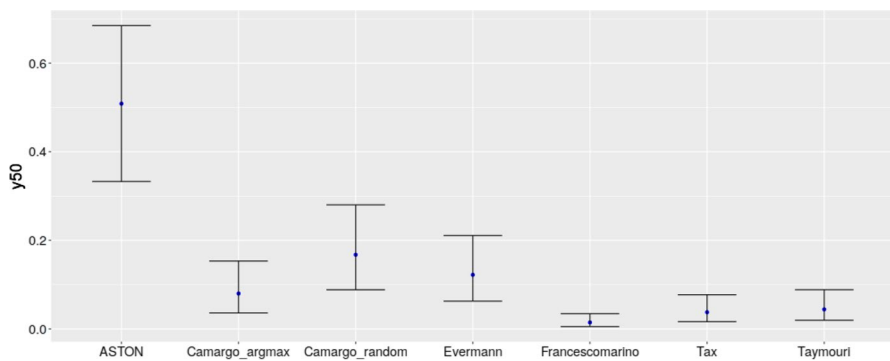
5.2 Results

Table 3 shows the results of the experimentation. ASTON obtains the best result in 8 out of 12 event logs, the second best result in 3 out of 12, and the third best result in 1 out of 12. These results seem to indicate that ASTON is highly consistent in both event logs with shorter and longer traces: specifically, for the 8 logs in which

Table 3 Results of the 5-fold cross-validation for the activity suffix prediction task in terms of the Damerau-Levenshtein similarity

Event log	Camargo/ argmax	Camargo/ random	Evermann	Francesco- marino	Tax	Taymouri	ASTON
BPI 2012	0.1851	<i>0.3891</i>	<i>0.1986</i>	0.1321	0.1409	0.1867	0.3947
BPI 2012 A	0.6536	<i>0.6441</i>	0.5847	0.2871	0.4597	0.5420	<i>0.6371</i>
BPI 2012 C	0.2218	<i>0.4495</i>	0.2693	0.0883	0.1717	<i>0.3557</i>	0.4544
BPI 2012 O	0.6845	<i>0.6042</i>	0.5544	0.4591	0.4972	0.5150	<i>0.6268</i>
BPI 2012 W	0.1941	<i>0.3110</i>	0.2800	0.1410	0.0975	0.2493	0.3993
BPI 2012 W C	0.0501	0.3191	<i>0.3372</i>	0.1126	0.0789	<i>0.3201</i>	0.3622
BPI 2013 C. P	<i>0.6641</i>	0.5337	<i>0.6416</i>	0.5276	0.5824	0.4358	0.6932
BPI 2013 I	0.2607	<i>0.5294</i>	0.4730	0.3607	0.3336	0.2452	0.6218
Env. permit	<i>0.8440</i>	0.7595	0.5713	0.3924	0.8163	0.4271	0.8469
Helpdesk	0.9110	0.8524	0.8354	0.4619	0.8695	0.6298	<i>0.8999</i>
Nasa	—	—	0.1218	0.0842	0.2320	0.4844	<i>0.4529</i>
Sepsis	—	—	0.2693	0.0742	0.1158	<i>0.3217</i>	0.3966
Avg Rank	3.93	2.33	2.99	6.92	5.57	5.25	1.02
Prob (%)	8.50	17.35	12.72	1.67	4.10	4.78	50.88

The best, second-best, and third-best results are highlighted in bold, italic, and bolditalic, respectively. Last two rows show the Plackett-Luce rankings and posterior probabilities

**Fig. 8** Credible intervals and the expected probability of winning for the tested approaches

it obtains the best results, there exists a significant difference in performance when comparing with the second-best approach in each log—ranging from 0.0049 until 0.0924, with an average of 0.0404—; whereas for the other 4 logs, it obtains results that are very close to the best approach in each log—with an average difference of 0.0292—.

The Plackett-Luce ranking results are also shown in Table 3. ASTON obtains the best overall ranking and the highest probability of being the best approach —

50.88%—, with a difference of 33.53 percentage points over the second approach —Camargo/random—. Furthermore, Fig. 8 shows the credible intervals —quantiles 5% and 95%— and the expected probability of being the best approach. In this figure, a non-overlap of the credible intervals of a pair of approaches means that they are statistically different for the given significance level. As shown in Fig. 8, ASTON does not overlap any other approach, showing that the results of ASTON are statistically different from those of the other tested approaches in terms of overall ranking. Note that the statistical tests are performed without taking into account the event logs *Sepsis* and *Nasa* since the approach of Camargo cannot be executed in those event logs.

To better quantify the differences between ASTON and the second and third-best approaches of the Plackett-Luce ranking —Camargo/random and Evermann— a hierarchical Bayesian test has been performed. Given a pair of approaches A and B , this test obtains the probability that A is better than B ($A > B$), that A has the same performance as B ($A = B$), and that A is worse than B ($A < B$). Thus, the approach A is better than B if $A > B$ is statistically significant, and, equivalently, the approach B is no worse than A if the sum of $B > A$ and $A = B$ is statistically significant. In this test, it is considered that a probability greater than 95% indicates statistical significance. The results of this hierarchical Bayesian test are shown in Table 4.

- Table 4 shows that ASTON significantly outperforms Camargo/random in six event logs —*BPI 2012 W*, *BPI 2012 W C.*, *BPI 2013 C.P.*, *BPI 2013 I.*, *Env. permit*, and *Helpdesk*—, being close to be significantly better in *BPI 2012 O.* —with 89.90% of probability—. For the other three event logs, there is a high

Table 4 Hierarchical bayesian tests comparing ASTON (A) against Camargo/(random) (C) and Evermann (E)

	ASTON vs Camargo/random			ASTON vs Evermann		
	$A > C$ (%)	$A = C$ (%)	$A < C$ (%)	$A > E$ (%)	$A = E$ (%)	$A < E$ (%)
BPI-2012	48.70	2.64	48.65	99.98	0	0.02
BPI-2012-A	15.05	7.20	77.75	99.98	0	0.02
BPI-2012-C	29.71	6.35	69.66	100	0	0
BPI-2012-O	89.90	0.35	9.74	99.98	0	0.02
BPI-2012-W	97.82	0.43	1.74	99.92	0.03	0.05
BPI-2012-W-C	99.26	0.05	0.68	87.51	1.04	11.44
BPI-2013-C-P	99.27	0.15	0.58	98.48	0.22	1.3
BPI-2013-I	99.98	0.02	0	100	0	0
Env-permit	99.89	0.02	0.09	100	0	0
Helpdesk	99.99	0	0.01	99.9	0	0.01
Nasa	–	–	–	100	0	0
Sepsis	–	–	–	99.98	0	0.02
Overall	99.4	0	0.6	100	0	0

Statistical significant results (greater than 95%) are underlined

degree of uncertainty to make any conclusion: Camargo/random does not significantly outperform ASTON in any event log, being close in *BPI 2012 A* and *BPI 2012 C*. —with 77.75% and 69.66% of probability, respectively—. In general, ASTON significantly outperforms the approach from Camargo et al., by a 99.4%. These results suggest that ASTON outperforms Camargo/random in event logs that have shorter traces, while it does not underperform Camargo/random when traces are longer, showing that ASTON is overall more robust to the underlying characteristics of the event log.

- Table 4 also shows that ASTON significantly outperforms the Evermann approach for every event log, except for *BPI 2012 W C.*, in which ASTON is close to achieve statistical significance with respect to Evermann —87.51% of probability—. Furthermore, ASTON, overall, significantly outperforms Evermann with an 100%. Therefore, an encoder-decoder learning approach with an attention mechanism is an effective method for the activity suffix prediction due to less accumulation of errors during the prediction phase.

5.3 Inference strategy comparison

We have compared the performance of ASTON using different inference strategies: selecting the most likely activity —*argmax*—, randomly sampling the next activity following the probability distribution —*random*—, using an unnormalized beam search strategy —*beam*—, and normalizing the beam search log probabilities according to the length of the predicted suffix —*beam norm.*—. The evaluation considers both the quality of the suffixes generated, measured by the Damerau-Levenshtein edit distance, and the length of the generated suffixes, providing insights into how the sampling strategy behaves with respect to the tested event log. Table 5

Table 5 Comparative analysis using normalized Damerau-Levenshtein performance and average predicted suffix length for various inference strategies

Event log	Normalized DL				Average Predicted Suffix Length				
	Arg	Rand	Beam	B.norm	Truth	Arg	Rand	Beam	B.norm
BPI 2012	0.177	0.384	0.377	0.395	19.44	141.12	16.51	4.93	19.08
BPI 2012 A	0.577	0.566	0.636	0.637	2.21	2.70	2.36	1.32	1.34
BPI 2012 C	0.234	0.420	0.420	0.454	10.90	72.18	9.93	3.36	5.55
BPI 2012 O	0.624	0.550	0.625	0.627	3.42	2.44	3.34	2.31	2.38
BPI 2012 W	0.151	0.361	0.350	0.399	15.55	123.90	12.81	3.20	10.43
BPI 2012 W.C	0.108	0.307	0.284	0.362	6.77	63.85	5.95	0.98	4.87
BPI 2013 C.P	0.696	0.630	0.691	0.693	2.86	1.26	2.24	1.11	1.14
Helpdesk	0.900	0.833	0.900	0.900	1.98	1.64	2.08	1.63	1.64
Sepsis	0.392	0.372	0.346	0.397	11.11	18.26	9.86	4.02	6.49
BPI 2013 I	0.540	0.585	0.612	0.622	7.18	28.47	6.22	2.76	3.18
Env. permit	0.847	0.766	0.847	0.847	2.88	2.49	2.87	2.47	2.47
Nasa	0.448	0.430	0.421	0.453	14.40	13.62	13.86	9.36	14.14

Bold denotes the best performing sampling strategy

shows the edit distance performance and the average predicted suffix length for each strategy.

The analysis from Table 5, echoing findings from [37], shows that the *argmax* strategy excels in logs with shorter traces (e.g., *BPI 2012 A, Helpdesk*), whereas the *random* strategy is more effective in logs with longer traces (e.g., *BPI 2012, Sepsis*). Additionally, the *argmax* strategy tends to predict longer suffixes than the ground truth in logs with longer traces, a limitation mitigated by the adaptability of the *random* strategy, which more effectively introduces the end of case token (e_{end}) in its predictions. Conversely, the *unnormalized beam* strategy shows steady performance across different logs, outperforming *argmax* in logs with longer traces and competing with the *random* strategy. It achieves comparable results to *argmax* in logs with shorter traces and bests the *random* approach. However, its tendency to predict shorter suffixes than needed in certain logs with longer traces (e.g., *BPI 2012 W, Sepsis*), as shown in Table 5, detracts from its effectiveness in those logs.

Lastly, the *normalized beam* strategy significantly enhances the *unnormalized beam* strategy. The length normalization of the log probabilities in the beam search alleviates the issue of predicting excessively short suffixes and enhances the overall quality of the activity suffix predictions. As shown in Table 5, this strategy offers the most consistent performance across all tested event logs, surpassing all other strategies except for *BPI 2013 C. P.* and *Helpdesk*, where *argmax* and *beam* strategies, respectively, perform better. Additionally, the *normalized beam* strategy outperforms the *unnormalized beam* strategy in all tested event logs in terms of average predicted suffix length (Table 5). This demonstrates that the normalization strategy effectively mitigates the issue of predicting shorter suffixes.

6 Conclusions and future work

This paper introduces ASTON, a novel approach for predictive process monitoring that uses an encoder-decoder architecture with attention to predict the next activity suffix given a running case. Unlike existing methods, ASTON is trained to predict the entire future suffix, reducing compounding errors. It leverages process model information during training and employs a beam search strategy with normalization to balance exploration and exploitation during inference. ASTON was extensively evaluated on 12 real-life event logs, significantly outperforming 6 state-of-the-art approaches from a statistical perspective. It secured the top result in 8 logs, second-best in 3, and third-best in 1, demonstrating robustness across a variety of processes.

For future work we will focus on enhancing the training and inference phases. For training, different attention operations will be explored to better capture long activity dependencies, and decoder-side attention mechanisms inspired by transformers will be incorporated. For inference, new sampling techniques will be investigated to further improve the selection of optimal suffix activities, improving the efficiency and accuracy of the model. Additionally, investigating individual prediction cases through case studies or specific examples could enhance our understanding of model behavior in practical scenarios and improve the interpretability of complex neural

network architectures. We aim to develop new methods for visualizing and explaining the predictions made by models that incorporate both GCN and RNN elements.

Acknowledgements This work has received financial support from the Consellería de Educación, Universidade e Formación Profesional (accreditation 2019-2022 ED431G-2019/04), the European Regional Development Fund (ERDF), which acknowledges the CiTIUS - Centro Singular de Investigación en Tecnoloxías Intelixentes da Universidade de Santiago de Compostela as a Research Center of the Galician University System, and the Spanish Ministry of Science and Innovation (grants PDC2021-121072-C21, PID2020-112623GB-I00, and TED2021-130374B-C21). Furthermore, the authors also wish to thank the supercomputer facilities provided by CESGA.

Author Contributions E. R-M: Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Original Draft, J.V: Conceptualization, Resources, Writing—Review & Editing, Visualization, Supervision, Project administration, Funding acquisition, Investigation M.L: Conceptualization, Resources, Writing—Review & Editing, Visualization, Supervision, Project administration, Funding acquisition, Investigation P. M-L Software, Validation, Investigation, Data Curation.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Aalst WMP (2016) Process mining: data science in action, 2nd edn. Springer, Cham
2. Francescomarino CD, Ghidini C (2022) Predictive process monitoring. Process mining handbook, vol 448. Lecture Notes in Business Information Processing. Springer, Cham, pp 320–346
3. Francescomarino CD, Dumas M, Maggi FM, Teinemaa I (2019) Clustering-based predictive process monitoring. *IEEE Trans Serv Comput* 12(6):896–909
4. Lee WLJ, Parra D, Munoz-Gama J, Sepúlveda M (2018) Predicting process behavior meets factorization machines. *Expert Syst Appl* 112:87–98
5. Leontjeva A, Conforti R, Francescomarino CD, Dumas M, Maggi FM (2015) Complex symbolic sequence encodings for predictive monitoring of business processes. In: 13th international conference on business process management (BPM 2015), vol 9253. pp 297–313
6. Leoni M, Aalst WMP, Dees M (2016) A general process mining framework for correlating, predicting and clustering dynamic behavior based on event logs. *Inf Syst* 56:235–257
7. Tax N, Teinemaa I, Zelst SJ (2020) An interdisciplinary comparison of sequence modeling methods for next-element prediction. *Softw Syst Model* 19(6):1345–1365
8. Pasquadibisceglie V, Appice A, Castellano G, Malerba D (2019) Using convolutional neural networks for predictive process analytics. In: 2019 international conference on process mining (ICPM 2019), pp 129–136
9. Mauro ND, Appice A, Basile TMA (2019) Activity prediction of business process instances with inception CNN models. In: XVIIIth international conference of the Italian association for artificial intelligence (AI*IA 2019), vol 11946. pp 348–361
10. Venugopal I, Töllich J, Fairbank M, Scherp A (2021) A comparison of deep-learning methods for analysing and predicting business processes. In: 2021 international joint conference on neural networks (IJCNN 2021), pp 1–8
11. Rama-Maneiro E, Vidal JC, Lama M (2024) Embedding graph convolutional networks in recurrent neural networks for predictive monitoring. *IEEE Tran Knowl Data Eng* 36(1):137–151
12. Taymouri F, Rosa ML, Erfani SM (2021) A deep adversarial model for suffix and remaining time prediction of event sequences. In: 2021 SIAM international conference on data mining (SDM 2021), pp 522–530

13. Mehdiyev N, Evermann J, Fettek P (2020) A novel business process prediction model using a deep learning method. *Bus Inf Syst Eng* 62(2):143–157
14. Ketykó I, Mannhardt F, Hassani M, Dongen BF (2022) What averages do not tell: predicting real life processes with sequential deep learning. In: 37th ACM/SIGAPP symposium on applied computing (SAC 2022), pp 1128–1131
15. Jalayer A, Kahani M, Beheshti A, Pourmasoumi A, Motahari-Nezhad HR (2020) Attention mechanism in predictive business process monitoring. In: 24th IEEE international enterprise distributed object computing conference (EDOC 2020), pp 181–186
16. Khan A, Le H, Do K, Tran T, Ghose A, Dam HK, Sindhgatta R (2021) Deepprocess: supporting business process execution using a MANN-based recommender system. In: 19th international conference on service-oriented computing (ICSOC 2021), vol 13121. pp 19–33
17. Tax N, Verenich I, Rosa ML, Dumas M (2017) Predictive business process monitoring with LSTM neural networks. In: 29th international conference on advanced information systems engineering (CAiSE 2017), vol 10253, pp 477–492
18. Evermann J, Rehse J, Fettek P (2017) Predicting process behaviour using deep learning. *Decis Support Syst* 100:129–140
19. Francescomarino CD, Ghidini C, Maggi FM, Petrucci G, Yeshchenko A (2017) An eye into the future: leveraging a-priori knowledge in predictive business process monitoring. In: 15th international conference on business process management (BPM 2017), vol 10445, pp 252–268
20. Weinzierl S, Dunzer S, Zilker S, Matzner M (2020) Prescriptive business process monitoring for recommending next best actions. In: Business process management forum 2020, vol 392. pp 193–209
21. Camargo M, Dumas M, Rojas OG (2019) Learning accurate LSTM models of business processes. In: 17th international conference on business process management (BPM 2019), vol 11675. pp 286–302
22. Dalmas B, Baranski F, Cortinovis D (2021) Predicting process activities and timestamps with entity-embeddings neural networks. In: 15th international conference on research challenges in information science (RCIS 2021), vol 415. pp 393–408
23. Sun X, Ying Y, Yang S, Shen H (2021) Remaining activity sequence prediction for ongoing process instances. *Int J Softw Eng Knowl Eng* 31(11 & 12):1741–1760
24. Lin L, Wen L, Wang J (2019). MM-Pred: a deep predictive model for multi-attribute event sequence. In: 2019 SIAM international conference on data mining (SDM 2019), pp 118–126
25. Venkateswaran P, Muthusamy V, Isahagian V, Venkatasubramanian N (2021) Robust and generalizable predictive models for business processes. In: 19th international conference on business process management (BPM 2021), vol 12875. pp 105–122
26. Agarwal P, Gupta A, Sindhgatta R, Dechu S (2022) Goal-oriented next best activity recommendation using reinforcement learning. *CoRR abs/2205.03219*
27. Arjovsky M, Bottou L, Gulrajani I, Lopez-Paz D (2019) Invariant risk minimization. *CoRR abs/1907.02893*
28. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: 30th annual conference on neural information processing systems (NIPS 2017), pp 5998–6008
29. Taymouri F, Rosa M.L, Erfani SM, Bozorgi ZD, Verenich I (2020) Predictive business process monitoring via generative adversarial nets: the case of next event prediction. In: 18th international conference on business process management (BPM 2020), vol 12168. pp 237–256
30. Weinzierl S (2021) Exploring gated graph sequence neural networks for predicting next process activities. In: 5th international workshop on artificial intelligence for business process management (AI4BPM2021)
31. Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: 5th international conference on learning representations (ICLR 2017). <https://openreview.net/forum?id=SJU4ayYgl>
32. Ruiz L, Gama F, Ribeiro A (2020) Gated graph recurrent neural networks. *IEEE Trans Signal Process* 68:6303–6318
33. Cho K, Merriënboer B, Gülçehre Ç, Bahdanau D, Bougares F, Schwenk H, Bengio Y (2014) Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: 2014 conference on empirical methods in natural language processing (EMNLP 2014), pp 1724–1734
34. Goodfellow I, Bengio Y, Courville A (2016) Deep learning. MIT Press, Cambridge

35. Bahdanau D, Cho K, Bengio Y (2015) Neural machine translation by jointly learning to align and translate. In: 3rd international conference on learning representations (ICLR 2015)
36. Singh D, Singh B (2020) Investigating the impact of data normalization on classification performance. *Appl Soft Comput* 97(Part B):105524
37. Rama-Maneiro E, Vidal J, Lama M (2022) Deep learning for predictive business process monitoring: review and benchmark. *IEEE Trans Serv Comput* 16(1):739–756
38. Bengio S, Vinyals O, Jaitly N, Shazeer N (2015) Scheduled sampling for sequence prediction with recurrent neural networks. In: 28th annual conference on neural information processing systems (NIPS 2015), pp 1171–1179
39. Wu Y, Wu Y, Schuster M, Chen Z, Le QV, Norouzi M, Macherey W, Krikun M et al (2016) Google's neural machine translation system: bridging the gap between human and machine translation. *CoRR* **abs/1609.08144**
40. Gali N, Mariescu-Istodor R, Hostettler D, Fränti P (2019) Framework for syntactic string similarity measures. *Expert Syst Appl* 129:169–185
41. Calvo B, Ceberio J, Lozano JA (2018) Bayesian inference for algorithm ranking analysis. In: 2018 genetic and evolutionary computation conference companion (GECCO 2018), pp 324–325
42. Benavoli A, Corani G, Demsar J, Zaffalon M (2017) Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis. *J Mach Learn Res* 18:77–17736
43. Calvo B, Santafé G (2016) scmamp: statistical comparison of multiple algorithms in multiple problems. *R J* 8(1):248–256

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.